

Diseño lógico de un Ensamblador de Tramas de Telemetría basado en FSM

Paul A. Landeo A., Ana L. I. Díaz Y., Gerardo A. Moreno M., Joseph D. Terrones L.

E.P. Ingeniería de Telecomunicaciones
Universidad Nacional Mayor de San Marcos
Lima, Perú

Resumen—El presente artículo documenta el diseño, la implementación y la validación experimental de un Ensamblador de Tramas de Telemetría implementado en lógica reconfigurable, orientado a su despliegue en nodos satelitales. La arquitectura propuesta extrae la carga de procesamiento (*offloading*) de estructuración de tramas desde un microcontrolador convencional hacia un hardware dedicado sobre una FPGA Cyclone IV EP4CE115F29C7N (placa de desarrollo DE2-115), evitando así la latencia y el *jitter* inherentes a la ejecución secuencial de instrucciones y a las rutinas de servicio de interrupción. El sistema captura de forma cíclica la información proveniente de tres sensores I2C (MPU6050, BME280 y BH1750), la encapsula en una trama de 29 bytes provista de cabecera de sincronización, identificador de misión y byte de redundancia cíclica, y la serializa mediante una interfaz UART asíncrona. Dado que la alta densidad del dispositivo seleccionado (114 480 elementos lógicos) incrementa la sección transversal de silicio expuesta a radiación ionizante, la unidad de control se implementó mediante tres máquinas de estados finitos idénticas sometidas a votación mayoritaria (Redundancia Triple Modular, TMR), con el propósito de enmascarar los *Single Event Upsets* (SEUs). La validación experimental, realizada con un osciloscopio digital Keysight DSOX1202G, confirmó una latencia de activación y una latencia extremo a extremo de 40.0 ns con una desviación nula respecto al modelo teórico, así como una tasa de transmisión física de 115 207 bps frente a los 115 200 bps nominales (0.005 % de error instrumental). Los resultados evidencian un determinismo temporal estricto, un consumo de recursos lógicos inferior al 1.3 % del dispositivo y una resiliencia estructural frente a fallas transitorias, validando la viabilidad del hardware dedicado tolerante a fallos.

Index Terms—FPGA, Redundancia Modular Triple (TMR), Máquina de Estados Finitos (FSM), Single Event Upset (SEU), Telemetría, Protocolo I2C, UART.

I. INTRODUCCIÓN

I-A. Problema y Motivación

En el contexto de las redes de sensores destinadas a la telemetría en entornos críticos, tales como los enlaces satelitales, la captura asíncrona de variables físicas por sensores periféricos plantea un desafío técnico. El envío de estos datos en su estado original a través del canal de radiofrecuencia no resulta factible debido al incremento en la tasa de error de bit (BER), la desincronización del receptor y la ocurrencia de colisiones. Por ello, resulta indispensable encapsular la carga útil (*payload*) en estructuras de tramas que incorporen redundancia cíclica, identificadores y cabeceras de sincronización para preservar la integridad del enlace [1].

Tradicionalmente, la estructuración de estas tramas se realiza mediante software en microcontroladores (MCU), los cuales consisten en hardware con una estructura fija diseñada para ejecutar instrucciones de manera estrictamente secuencial. Este método genera limitaciones críticas en aplicaciones que requieren baja latencia y alta fiabilidad; la dependencia de rutinas de servicio de interrupción (ISR) provoca *jitter* temporal derivado del cambio de contexto y la limpieza de *pipelines* [2]. La CPU debe alternar mediante multiplexación temporal entre la captura I2C, el cálculo de paridad y la transmisión UART, bloqueando el procesamiento y elevando la probabilidad de desbordamiento.

Asimismo, la robustez frente a la radiación en arquitecturas CMOS comerciales es deficiente [3]. Las partículas ionizantes inducen *Single Event Upsets* (SEUs) que alteran nodos de memoria, corrompen el contador de programa y desvían las transiciones de la máquina de estados, causando fallos funcionales [4], [5].

I-B. Trabajos Relacionados

El análisis comparativo entre FPGA y MCU [1] evidencia que la naturaleza paralela de las FPGA permite instanciar periféricos como bloques de silicio independientes. El estudio de [2] formaliza el compromiso entre *throughput* y latencia, señalando que la reducción del *jitter* exige abandonar esquemas basados en interrupciones por software. En plataformas orbitales, [3] resalta la viabilidad del procesamiento determinista en FPGA frente a los MCU.

Para abordar la tolerancia a fallos, [4] propone arquitecturas *lockstep* temporales, y [5] caracteriza nodos endurecidos por radiación. Finalmente, [6] y [7] establecen los métodos de mitigación de errores en máquinas de estados finitos mediante Redundancia Triple Modular (TMR), base teórica de la mitigación aplicada en esta investigación.

I-C. Objetivos

El objetivo general es diseñar e implementar una arquitectura digital estructuradora de tramas en hardware dedicado, basada en Máquinas de Estados Finitos (FSM), decodificadores libres de fallos y bloques de memoria, extrayendo la carga de procesamiento (*offloading*) de los microcontroladores para garantizar paralelismo real y latencia ultrabaja.

Los objetivos específicos comprenden:

El código Verilog fue sintetizado mediante Quartus Prime para la FPGA Cyclone IV EP4CE115F29C7N de la tarjeta

Tabla I
ESTRUCTURA SECUENCIAL Y DISTRIBUCIÓN DE CAMPOS DE LA TRAMA DE TELEMETRÍA DE 29 BYTES

Índice	Segmento	Valor Hexadecimal	Origen Físico	Función del Sistema
0	Frame Header	0xAA	ROM Estática	Sincronización del Receptor (SYNC1)
1	Frame Header	0x55	ROM Estática	Sincronización del Receptor (SYNC2)
2	Frame Header	0x01	ROM Estática	Identificador de Identidad (DEVICE_ID)
3	Frame Header	0x18	ROM Estática	Longitud Útil del Payload (24d)
4-17	Payload RAM	Dinámico	Sensor I2C MPU6050	Acelerómetro, Giroscopio y Temperatura (14 B)
18-25	Payload RAM	Dinámico	Sensor I2C BME280	Presión, Temperatura y Humedad (8 B)
26-27	Payload RAM	Dinámico	Sensor I2C BH1750	Sensor de luz ambiental / Lux (2 B)
28	Checksum Val	Dinámico (XOR)	Hardware Aritmético	Redundancia cíclica dinámica

Tabla II
TABLA DE ANÁLISIS DE SEÑALES DE ENTRADA Y SALIDAS DE CONTROL DEL ALGORITMO FSM

Entradas (Condiciones Lógicas Hacia FSM)	Salidas (Acciones de Control Hacia Datapath)
data_ready_pulse: Pulso fin de captura I2C (192 bits). txb_active: Bandera UART ocupada. txb_falling: Detector UART libre. ptr_reg == 5'd3 (MAX_ROM): Límite cabecera. ptr_reg == 5'd27 (MAX_RAM): Límite payload.	we_ram: Habilitación escritura RAM M9K. sel_rom: MUX RAM/ROM. load_tx: Disparo para captura y serialización. ptr[4:0]: Dirección física sincrónica. clear_chk / update_chk: Control aritmético.

Tabla III
TRANSICIONES PRINCIPALES

Estado	Condición	Siguiente
IDLE	I2C_RDY = 1	WRITE_RAM
WRITE_RAM	Incondicional	TX_ROM
TX_ROM	MAX_ROM = 1	TX_RAM
TX_RAM	MAX_RAM = 1	TX_STATUS
TX_STATUS	UB = 1	TX_CHKSUM
TX_CHKSUM	Incondicional	DONE
DONE	Incondicional	IDLE

Tabla IV
TABLA DE VERDAD DE SALIDAS (MEALY)

Estado	WR_EN	MUX	TX_ON	INC	CLR
IDLE	0	0	0	0	1
WRITE_RAM	1	0	0	0	0
TX_ROM	0	0	1	1	0
TX_RAM	0	0	1	1	0
TX_STATUS	0	0	1	1	0
TX_CHKSUM	0	1	1	0	0
DONE	0	0	0	0	0

DE2-115 [8]. Su alta densidad (114 480 LEs) justifica el uso del TMR ante la probabilidad estadística de impacto por radiación en sus compuertas.

III-B. Estado Operativo y Conexiones Físicas

El bitstream (.sof) se cargó en la placa. Las Figuras 5, 6 y 7 evidencian el estado activo, la conexión del módulo TTL-USB

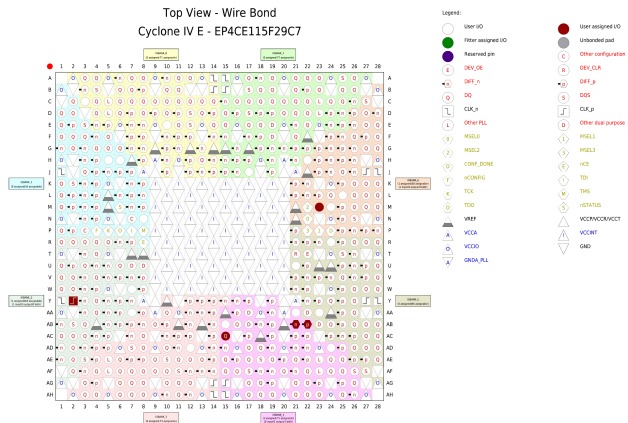


Figura 4. Asignación en el Pin Planner de Quartus Prime.

Tabla V
ASIGNACIÓN DE PINES FÍSICOS

Señal Lógica	Pin FPGA	Función
clk_50M	PIN_Y2	Entrada global (Reloj).
rst_n	PIN_M23	Reset asíncrono (KEY0).
i2c_scl	PIN_AB22	GPIO (Open-Drain).
i2c_sda	PIN_AC15	GPIO (Open-Drain).
uart_tx	PIN_AB21	Salida asíncrona TX.

y el bus I2C en cascada.

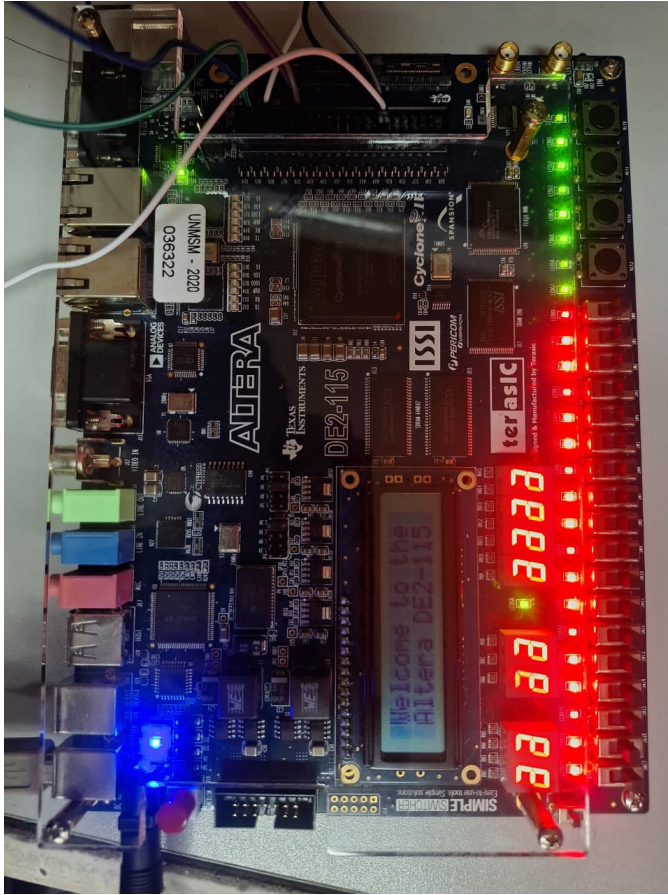


Figura 5. Tarjeta DE2-115 energizada con el bitstream TMR.

IV. RESULTADOS EXPERIMENTALES Y ANÁLISIS

IV-A. Síntesis RTL y Verificación Lógica

La Figura 8 documenta el ruteo interno (Datapath vs. Control Path), y la Figura 9 valida la morfología de la máquina generada.

La verificación en ModelSim (Figuras 10 y 11) comprobó el flujo nominal y la respuesta de la lógica TMR ante un reinicio abrupto inyectado a medio flujo, garantizando el retorno seguro a IDLE sin sobrescritura de M9K.

IV-B. Evaluación Física de Latencias

El osciloscopio digital auditó el silicio físico. La Figura 12 valida 40.0 ns de Latencia de Activación, y la Figura 13 demuestra la ausencia de *dead time* en la Latencia End-to-End. La Figura 14 verifica el ancho de bit de 8.68 μ s.

La Tabla VI consolida la comparación sin cortes ni desbordamientos, estructurando el comportamiento matemático contra el térmico-físico.

La Figura 15 corrobora la recepción íntegra sin *Packet Loss* en la terminal.

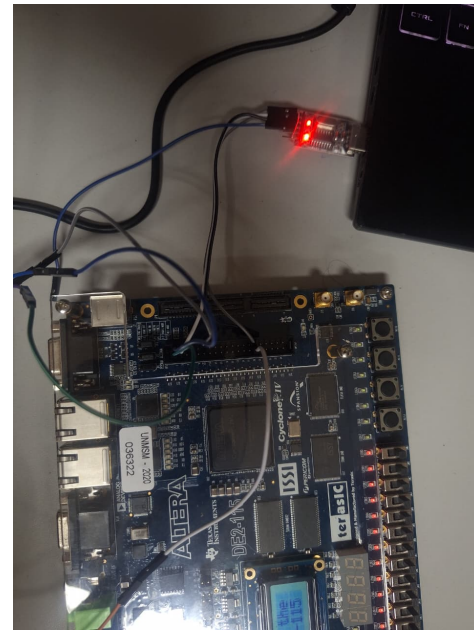


Figura 6. Acople del transceptor UART a los terminales GPIO.

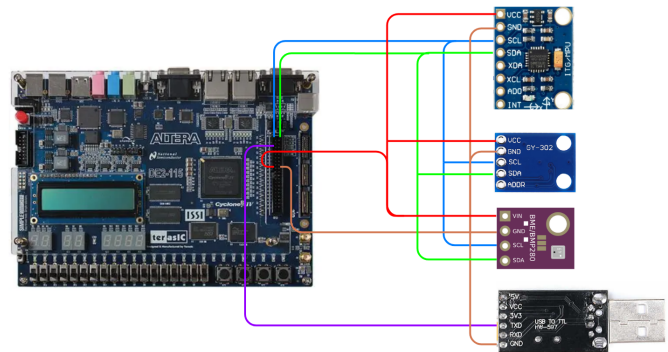


Figura 7. Topología eléctrica de la cascada de sensores I2C multiplexados.

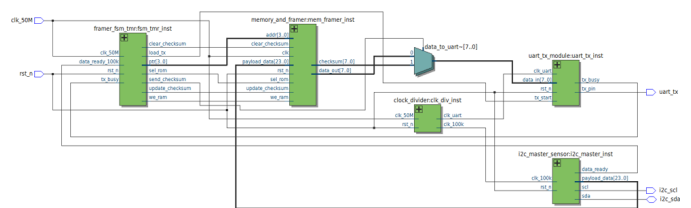


Figura 8. Diagrama RTL inferido, demostrando separación de bus de 192 bits.

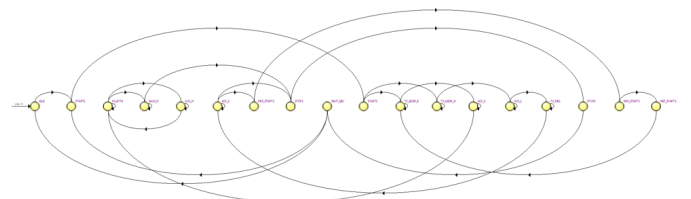


Figura 9. Visor FSM validando los estados lógicos sintetizados.

Tabla VI
MATRIZ DE COMPARACIÓN FUNCIONAL Y ANÁLISIS DE DESVIACIONES: MODELO MATEMÁTICO VS. SILICIO OPERATIVO

Métrica Evaluada	Valor Teórico (Simulación)	Medición Física	Desviación	Causa Raíz del Error y Justificación Técnica
Latencia de Activación FSM	40.0 ns (2 ciclos de clk_50M)	40.0 ns	0.0 ns (0 %)	Alineación Síncrona: Requiere ineludiblemente 2 flancos completos sin error diferencial en buffers.
Latencia UART Total	2.691 ms (31 B $\times$ 10 bits / 115200)	2.69 ms	1.0 μ s (0.048 %)	Cuantización Lógica: El divisor truncado genera 115207bps físicos acortando la ráfaga masiva.
Latencia End-to-End	40.0 ns (2 ciclos estructurales)	40.0 ns	0.0 ns (0 %)	Determinismo Estricto: El procesador reacciona sin retardos transitorios a pesar de computas TMR.
Ancho de Bit Físico	8.6805 μ s (Período: 1 / 115200)	8.68 μ s	0.5 ns (0.005 %)	Incertidumbre Instrumental: Fluctuación atribuible al muestreo del osciloscopio y ppm del cristal.

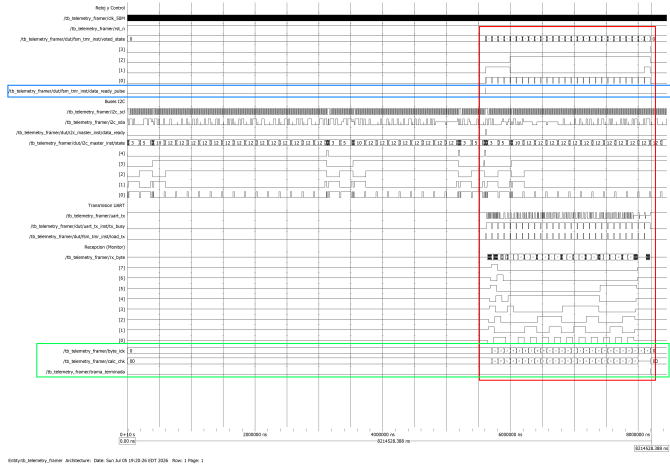


Figura 10. Formas de onda (Caso 1 Nominal): Flujo ininterrumpido.

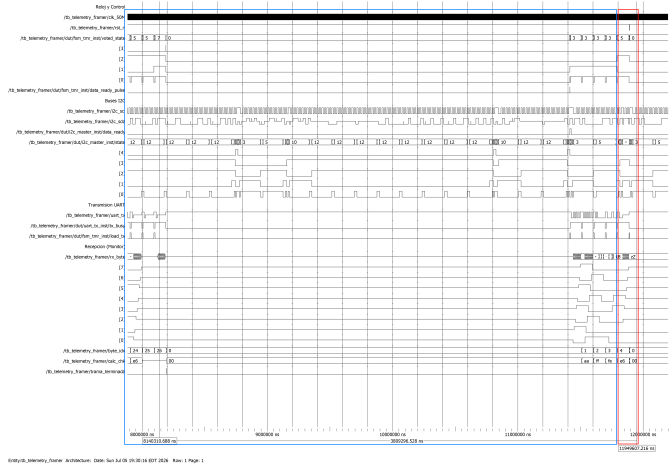


Figura 11. Formas de onda (Caso 2): Mitigación de reinicio abrupto y recuperación.



Figura 12. Oscilograma de Latencia de Activación (40.0 ns exactos).



Figura 13. Latencia End-to-End: cierre I2C (canal 1) e inicio TX (canal 2).



Figura 14. Validación del Baud Rate físico de la UART (8.68 μ s).

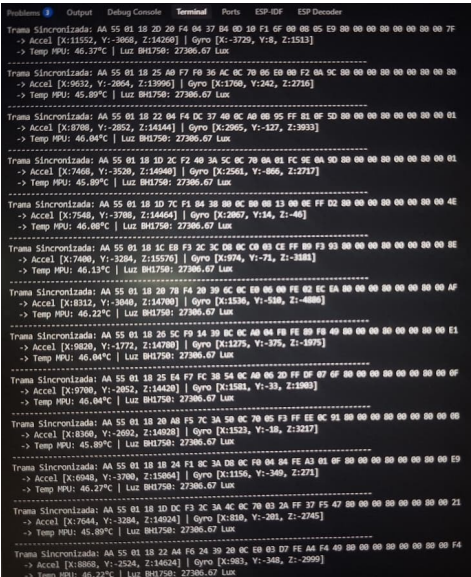


Figura 15. Recepción consolidada validando el hardware lógico M9K.

IV-C. Auditoría de Recursos Lógicos (Fitter)

A pesar del costo lógico de triplicar nodos e inyectar votadores mayoritarios, la estructura consumió solo el 1.31 % del EP4CE115, según detalla la Tabla VII.

Tabla VII
REPORTE ANALÍTICO DEL FITTER (OCUPACIÓN ESTÁTICA)

Recurso Sintetizado	Uso / Total	Utilización
Total Logic Elements (LEs)	1 502 / 114 480	1.31 %
– Combinacionales (LUTs)	1 483 / 114 480	1.30 %
– Registros Dedicados D	904 / 114 480	0.79 %
Terminales Físicos E/S (GPIO)	5 / 529	< 1.0 %
Bloques Dedicados M9K	1 / 432	< 1.0 %
Bits Globales en SRAM	512 / 3 981 312	< 1.0 %
Redes de Reloj Global	4 / 20	20.0 %

V. DISCUSIÓN ANALÍTICA

El rigor del análisis experimental dictamina incuestionablemente la superioridad categórica del procesamiento concurrente gobernado por estados sobre arquitecturas MCU clásicas. La evidencia primordial es la **Latencia End-to-End mantenida en 40.0 ns (0 % fluctuación)**. Al expurgar las Rutinas de Servicio de Interrupción (ISR), se erradica la raíz del *jitter* por cambios de contexto en memoria.

Las tolerancias observadas en radiotransmisión asíncrona (0.048 % diferencial global y 0.005 % en ancho de bit) obedecen a propiedades de truncación por divisores de onda ($\frac{50 \text{ MHz}}{115200 \text{ Hz}} = 434,027$). Forzando la señal a 115,207 baudios, el hardware sintetiza una red tolerada por normas internacionales, impidiendo disonancias en la recuperación.

El verdadero cuello de botella latente en la red sensora no es la FPGA, sino los retardos intrínsecos de los COTS. El *Clock Stretching* originado por la conversión Análogo-Digital del BME280 y MPU6050 frena el tren de reloj `i2c_scl`, imponiendo una tasa máxima ineludible al ancho de banda del subsistema.

VI. CONCLUSIONES Y TRABAJOS FUTUROS

VI-A. Conclusiones

La arquitectura propuesta demuestra el éxito de la extracción (*Offloading*) de estructuración de tramas hacia hardware dedicado en FPGA, asegurando latencias ultrabajas y rígidamente deterministas exentas de *jitter*.

El diseño del decodificador combinacional puro fungió como eslabón vital, logrando aislar de forma perfecta la Máquina de Estados respecto a la memoria estática M9K, sin agotar ciclos de reloj adicionales por arbitraje de bus.

El blindaje estructural TMR provee una defensa lógica inquebrantable frente a inversiones de bit (SEU). Triplicar la lógica consumió apenas un imperceptible 1.31 % del Cyclone IV EP4CE115, certificando que se puede dotar a un diseño de inmunidad grado espacial dejando libre un monumental ~98 % del silicio para otras cargas de procesamiento complejas.

VI-B. Ruta de Escalabilidad e Investigaciones Proyectadas

Para escalar este diseño experimental hacia misiones satelitales, se trazan las siguientes pautas:

1. **Migración a Sustratos Rad-Hard[®]**: Sintetizar la lógica en familias tolerantes a radiación como SmartFusion2 (SRAM inalterable a SEU) o Zynq-7000 Space Grade, para certificar tolerancia a Dosis Total Ionizante (TID) en órbita LEO.
2. **Implementación EDAC**: Diseñar IP Cores de Detección y Corrección de Errores (Ej. códigos de Hamming o Reed-Solomon) en los puertos de la RAM M9K. El TMR protege el flujo de control, pero los 192 bits estáticos siguen susceptibles a corrupción de bit múltiple (*Multiple Cell Upsets*).
3. **Protocolización CCSDS**: Adecuar el decodificador de cabecera al formato *Space Packet Protocol* del CCSDS para estandarizar las telecomunicaciones con estaciones terrenas globales.
4. **Bus de Alta Velocidad**: Reformar los puentes I2C por redes seriales diferenciales o de alta velocidad (CAN-FD, SpaceWire) para superar el *Clock Stretching* y mitigar el ruido electromagnético.

REFERENCIAS

- [1] DYC Electronic, “FPGA vs microcontroller understanding the key differences and use cases,” *PCBA Manufacture in China*, May 22, 2026. [Online]. Available: <https://www.dyc-electronic.com/pt/fpga-vs-microcontroller/>.
- [2] NI, “More throughput vs. less latency: Understand the difference,” Nov. 1, 2013. [Online]. Available: <https://www.ni.com/>.
- [3] Thai, “Applications for FPGAs on nanosatellites,” M.S. thesis, Graduate Program in Earth and Space Science, York Univ., Toronto, ON, Canada, Apr. 2014.
- [4] R. Tedeschi *et al.*, “Temporal lockstep: Low-cost resilient design for microcontroller-class RISC-V processors,” *IEEE Access*, vol. 14, pp. 1–1, Mar. 2026.
- [5] M. Duni, “On the development of radiation-hardened high performance computing nodes for satellite systems,” Master’s thesis, Politecnico di Torino, Turin, Italy, Dec. 2025.
- [6] M. Kubica and R. Czerwinski, “Error mitigation methods for FSM using triple modular redundancy,” *Appl. Sci.*, vol. 15, no. 12, p. 6726, 2025.
- [7] D. Agiakatsikas, “High-level synthesis of triple modular redundant FPGA circuits with energy efficient error recovery mechanisms,” Ph.D. dissertation, UNSW, Sydney, NSW, Australia, June 2019.
- [8] Intel Corporation, “Cyclone IV E EP4CE115 FPGA,” Intel, 2026. [Online]. Available: <https://www.altera.com/products/fpga/cyclone/iv/e/ep4ce115-f29>.

APÉNDICE A
CÓDIGO FUENTE DEL MÓDULO TOP (TOP_TELEMETRY_FRAMER.V)

```
1 module top_telemetry_framer (
2     input wire clk_50M,
3     input wire rst_n,
4     inout wire i2c_sda,
5     output wire i2c_scl,
6     output wire uart_tx
7 );
8
9     wire clk_100k;
10    wire clk_uart;
11    wire data_ready;
12    wire [191:0] payload_data;
13    wire [4:0] mem_ptr;
14    wire sel_rom, we_ram, load_tx;
15    wire clear_chk, update_chk, send_chk;
16    wire [7:0] data_from_mem, checksum_val;
17    wire tx_busy;
18    wire [7:0] data_to_uart;
19
20    clock_divider clk_div_inst (
21        .clk_50M(clk_50M),
22        .rst_n(rst_n),
23        .clk_100k(clk_100k),
24        .clk_uart(clk_uart)
25    );
26
27    i2c_master_sensor i2c_master_inst (
28        .clk_100k(clk_100k),
29        .rst_n(rst_n),
30        .scl(i2c_scl),
31        .sda(i2c_sda),
32        .data_ready(data_ready),
33        .payload_data(payload_data)
34    );
35
36    memory_and_framer mem_framer_inst (
37        .clk(clk_50M),
38        .rst_n(rst_n),
39        .addr(mem_ptr),
40        .sel_rom(sel_rom),
41        .we_ram(we_ram),
42        .payload_data(payload_data),
43        .clear_checksum(clear_chk),
44        .update_checksum(update_chk),
45        .data_out(data_from_mem),
46        .checksum(checksum_val)
47    );
48
49    assign data_to_uart = send_chk ? checksum_val : data_from_mem;
50
51    framer_fsm_tmr fsm_tmr_inst (
52        .clk_50M(clk_50M),
53        .rst_n(rst_n),
54        .data_ready_100k(data_ready),
55        .tx_busy(tx_busy),
56        .we_ram(we_ram),
57        .sel_rom(sel_rom),
58        .load_tx(load_tx),
59        .ptr(mem_ptr),
60        .clear_checksum(clear_chk),
61        .update_checksum(update_chk),
62        .send_checksum(send_chk)
63    );
64
65    uart_tx_module uart_tx_inst (
66        .clk_uart(clk_uart),
67        .rst_n(rst_n),
68        .tx_start(load_tx),
69        .data_in(data_to_uart),
70        .tx_pin(uart_tx),
71        .tx_busy(tx_busy)
72    );
73
74 endmodule
```

Listing 1. top_telemetry_framer.v: Módulo superior estructurador.

APÉNDICE B
CÓDIGO FUENTE DEL BANCO DE PRUEBAS (TB_TELEMETRY_FRAMER.V)

```
1 `timescale 1ns/1ps
2 module tb_telemetry_framer;
3     reg clk_50M;
4     reg rst_n;
5     wire i2c_scl;
6     wire i2c_sda;
7     wire uart_tx;
8
9     top_telemetry_framer dut (
10         .clk_50M(clk_50M),
11         .rst_n(rst_n),
12         .i2c_sda(i2c_sda),
13         .i2c_scl(i2c_scl),
14         .uart_tx(uart_tx)
15     );
16
17     initial begin
18         clk_50M = 0;
19         forever #10 clk_50M = ~clk_50M;
20     end
21
22     reg tb_sda_out;
23     reg tb_sda_en;
24     assign i2c_sda = tb_sda_en ? tb_sda_out : 1'bz;
25
26     reg trama_terminada;
27
28     task wait_scl_negedge;
29         begin @(negedge i2c_scl); end
30     endtask
31
32     task send_i2c_byte(input [7:0] byte_data);
33         integer i;
34         begin
35             tb_sda_en = 1;
36             for (i=7; i>=0; i=i-1) begin
37                 tb_sda_out = byte_data[i];
38                 wait_scl_negedge();
39             end
40         end
41     endtask
42
43     always @(negedge rst_n) begin
44         disable sim_i2c_slave_block;
45         tb_sda_en = 0;
46         tb_sda_out = 1;
47     end
48
49     always @(posedge rst_n) begin : sim_i2c_slave_block
50         integer i;
51         forever begin
52             repeat (8) wait_scl_negedge();
53             tb_sda_en = 1; tb_sda_out = 0; wait_scl_negedge(); #50000; tb_sda_en = 0;
54             repeat (8) wait_scl_negedge();
55             tb_sda_en = 1; tb_sda_out = 0; wait_scl_negedge(); tb_sda_en = 0;
56             repeat (8) wait_scl_negedge();
57             tb_sda_en = 1; tb_sda_out = 0; wait_scl_negedge(); #20000;
58             for (i=0; i<13; i=i+1) begin
59                 send_i2c_byte(8'h10 + i);
60                 tb_sda_en = 0; wait_scl_negedge();
61             end
62             send_i2c_byte(8'h1D);
63             tb_sda_en = 0; wait_scl_negedge();
64             wait (i2c_scl == 1 && i2c_sda == 1); #5000;
65
66             repeat (8) wait_scl_negedge();
67             tb_sda_en = 1; tb_sda_out = 0; wait_scl_negedge(); #50000; tb_sda_en = 0;
68             repeat (8) wait_scl_negedge();
69             tb_sda_en = 1; tb_sda_out = 0; wait_scl_negedge(); tb_sda_en = 0;
70             repeat (8) wait_scl_negedge();
71             tb_sda_en = 1; tb_sda_out = 0; wait_scl_negedge(); #20000;
72             for (i=0; i<7; i=i+1) begin
73                 send_i2c_byte(8'h20 + i);
74                 tb_sda_en = 0; wait_scl_negedge();
75             end
76             send_i2c_byte(8'h27);
77             tb_sda_en = 0; wait_scl_negedge();
78             wait (i2c_scl == 1 && i2c_sda == 1); #5000;
79         end
80     end
81 endmodule
```



```

80     repeat(8) wait_scl_negedge();
81     tb_sda_en = 1; tb_sda_out = 0; wait_scl_negedge(); #50000; tb_sda_en = 0;
82     repeat(8) wait_scl_negedge();
83     tb_sda_en = 1; tb_sda_out = 0; wait_scl_negedge(); tb_sda_en = 0;
84     repeat(8) wait_scl_negedge();
85     tb_sda_en = 1; tb_sda_out = 0; wait_scl_negedge(); #20000;
86     send_i2c_byte(8'h30); tb_sda_en = 0; wait_scl_negedge();
87     send_i2c_byte(8'h31); tb_sda_en = 0; wait_scl_negedge();
88     wait(i2c_scl == 1 && i2c_sda == 1); #5000;
89 end
90 end
91
92 reg [7:0] rx_byte;
93 integer byte_idx = 0;
94 reg [7:0] calc_chk = 8'h00;
95
96 always begin
97     @(negedge uart_tx);
98     #4340;
99     if (uart_tx == 0) begin : read_uart_loop
100         integer j;
101         for (j=0; j<8; j=j+1) begin
102             #8680;
103             rx_byte[j] = uart_tx;
104         end
105         #8680;
106
107         if (byte_idx < 30) calc_chk = calc_chk ^ rx_byte;
108         $display("[%0t] RX Byte %0d = 0x%02X", $time, byte_idx, rx_byte);
109
110         if (byte_idx == 30) begin
111             trama_terminada = 1; byte_idx = 0; calc_chk = 8'h00;
112             #100; trama_terminada = 0;
113         end else begin
114             byte_idx = byte_idx + 1;
115         end
116     end
117 end
118
119 initial begin : test_cases
120     rst_n = 0; tb_sda_en = 0; tb_sda_out = 1; trama_terminada = 0; #200;
121
122     $display("[CASO 1] Flujo Nominal...");
123     rst_n = 1; wait(trama_terminada == 1); #10000;
124
125     $display("[CASO 2] Reinicio Abrupto...");
126     wait(byte_idx == 5); rst_n = 0; #500; byte_idx = 0; calc_chk = 0; #1000;
127
128     $display("[CASO 3] Perdida I2C...");
129     force dut.i2c_master_inst.data_ready = 1'b0; rst_n = 1; #500000;
130     release dut.i2c_master_inst.data_ready; #10000;
131
132     $display("[CASO 4] Saturacion TX...");
133     force dut.uart_tx_inst.tx_busy = 1'b1; wait(dut.fsm_tmr_inst.data_ready_100k == 1);
134     #50000; release dut.uart_tx_inst.tx_busy; wait(trama_terminada == 1);
135
136     $finish;
137 end
138 endmodule

```

Listing 2. tb_telemetry_framer.v: Banco de validación y testbench exhaustivo.